

Q2 / Q26 – FCFS Disk Scheduling

Requests: 55, 58, 39, 18, 90, 160, 150, 38, 184, Start Head Position: 50

Q29 – FCFS Disk Scheduling

Requests: 56, 59, 40, 19, 91, 161, 151, 39, 185, Start Head Position: 48

Q32- FCFS Disk Scheduling

Requests: 65, 95, 30, 91, 18, 116, 142, 44, 168 .Start Head Position: 52

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
void main()
```

```
{
```

```
    int req[20],temp,i,j,n,head,total=0,index,min;
```

```
    printf("Enter the no.of processes to be requested : ");
```

```
    scanf("%d",&n);
```

```
    printf("Enter the request : ");
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        scanf("%d",&req[i]);
```

```
    }
```

```
    printf("Enter the initial head position : ");
```

```
    scanf("%d",&head);
```

```
    printf("\nSeek Sequence = %d ",head);
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        total = total + abs(head-req[i]);
```

```
        head = req[i];
```

```
        printf("\t->%d",head);
```

```
    }
```

```
    printf("\nTotal Head Movement=%d",total);
```

```
}
```

Q11 – SSTF Disk Scheduling

Requests: 186, 89, 44, 70, 102, 22, 51, 124. Start Head Position: 70

Q28 – SSTF Disk Scheduling

Requests: 24, 90, 133, 43, 188, 70, 37, 55. Start Head Position: 58

```
#include<stdio.h>
#include<stdlib.h>
main()
{
    int req[20],temp,i,j,n,head,total=0,index,min;
    printf("Enter the no.of processes to be requested : ");
    scanf("%d",&n);
    printf("Enter the request : ");
    for(i=0;i<n;i++)
    {
        scanf("%d",&req[i]);
    }
    printf("Enter the initial head position : ");
    scanf("%d",&head);
    printf("\nSeek Sequence : %d",head);
    for(i=0;i<n;i++)
    {
        min = 1000;
        for(j=0;j<n;j++)
        {
            if ( (abs(head - req[j]))< min)
            {
                min = abs(head-req[j]);
                index = j;
            }
        }
    }
```

```
        total = total + abs(head-req[index]);
        head = req[index];
        req[index] = 1000;
        printf("-> %d",head);
    }
    printf("\nTotal Head Movement=%d",total);
}
```

Q9 – C-SCAN Disk Scheduling

Requests: 80, 150, 60, 135, 40, 35, 170. Start Head Position: 70. Direction: Right

Q14 – C-SCAN Disk Scheduling

Requests: 33, 99, 142, 52, 197, 79, 46, 65. Start Head Position: 72. Direction: Left. only change `printf("Direction: Left\n");`

Q25 – C-SCAN Disk Scheduling

Requests: 23, 89, 132, 42, 187, 69, 36, 55. Start Head Position: 40. Direction: Left. only change `printf("Direction: Left\n");`

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
void sort(int req[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; ++i) {
        for (j = i + 1; j < n; ++j) {
            if (req[i] > req[j]) {
                temp = req[i];
                req[i] = req[j];
                req[j] = temp;
            }
        }
    }
}
```

```
void scan(int req[], int n, int head, int maxc)
{
    int i, hm, index, total=0;
    printf("Direction: Right\n");

    for (i = 0; i < n; ++i) {
        if (req[i] > head) {
            index = i;
            break;
        }
    }

    total = total + (maxc - 1 - head);
    total = total + maxc - 1;
    head = 0;
    total = total + abs(head - req[index-1]);

    printf("\nTotal Head Movement=%d", total);
}
```

```
void main()
{
    int req[20],i,n,head,maxc;

    printf("Enter the no.of processes to be requested : ");
    scanf("%d",&n);

    printf("Enter the request : ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);

    sort(req,n);

    printf("\nSorted Request : ");
    for(i=0;i<n;i++)
        printf("\t%d",req[i]);

    printf("\nEnter the initial head position : ");
    scanf("%d",&head);

    printf("\nEnter Number of Cylinder : ");
    scanf("%d",&maxc);

    printf("\nSeek Sequence : %d",head);

    scan(req,n,head,maxc);
}
```

Q7 – SCAN Disk Scheduling

Requests: 86, 147, 91, 170, 95, 130, 102, 70. Start Head Position: 125. Direction: Left

Q19 – SCAN Disk Scheduling

Requests: 176, 79, 34, 60, 92, 11, 41, 114. Start Head Position: 65. Direction: Left

Use exact same program.

Q23 – SCAN Disk Scheduling

Requests: 33, 99, 142, 52, 197, 79, 46, 65. Start Head Position: 72. Direction: Right

Only change this line in the code: printf("Direction: Right\n");

```
#include<stdio.h>
#include<stdlib.h>
```

```
void sort(int req[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; ++i) {
        for (j = i + 1; j < n; ++j) {
            if (req[i] > req[j]) {
                temp = req[i];
                req[i] = req[j];
                req[j] = temp;
            }
        }
    }
}
```

```
void scan(int req[], int n, int head, int maxc)
{
    int i, hm, index, total=0;
    printf("Direction: Left\n");

    for (i = 0; i < n; ++i) {
        if (req[i] > head) {
            index = i;
            break;
        }
    }

    for (i = index-1; i >=0; i--) {
        hm = abs(req[i] - head);
        printf("Request %d served. Head movement: %d\n", req[i], hm);
        total = total + hm;
        head = req[i];
    }
```

```

    hm = abs(head - 0);
    total = total + hm;
    head = 0;
    printf("Reached to Left End, Head Movement : %d\n",hm);

    for (i = index; i < n; i++) {
        hm = abs(req[i] - head);
        printf("Request %d served. Head movement: %d\n", req[i], hm);
        total = total + hm;
        head = req[i];
    }

    printf("\nTotal Head Movement=%d",total);
}

void main()
{
    int req[20],i,n,head,maxc;
    printf("Enter the no.of processes to be requested : ");
    scanf("%d",&n);

    printf("Enter the request : ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);

    sort(req,n);

    printf("\nSorted Request : ");
    for(i=0;i<n;i++)
        printf("\t%d",req[i]);

    printf("\nEnter the initial head position : ");
    scanf("%d",&head);

    printf("\nEnter Number of Cylinder : ");
    scanf("%d",&maxc);

    printf("\nSeek Sequence : %d",head);

    scan(req,n,head,maxc);
}

```

Q13 – LOOK Disk Scheduling

Requests: 176, 79, 34, 60, 92, 11, 41, 114. Start Head Position: 65. Direction: Left

Q22 – LOOK Disk Scheduling

Requests: 23, 89, 132, 42, 187, 69, 36, 55. Start Head Position: 40. Direction: Left

```
#include<stdio.h>
#include<stdlib.h>
```

```
void sort(int req[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; ++i) {
        for (j = i + 1; j < n; ++j) {
            if (req[i] > req[j]) {
                temp = req[i];
                req[i] = req[j];
                req[j] = temp;
            }
        }
    }
}
```

```
void look(int req[], int n, int head)
{
    int i, hm, index, total=0;
    printf("Direction: Left\n");

    for (i = 0; i < n; ++i) {
        if (req[i] > head) {
            index = i;
            break;
        }
    }

    for (i = index-1; i >=0; i--) {
        hm = abs(req[i] - head);
        printf("Request %d served. Head movement: %d\n", req[i], hm);
        total = total + hm;
        head = req[i];
    }

    printf("Reached to Left End Last Request Served\n");

    for (i = index; i < n; i++) {
        hm = abs(req[i] - head);
        printf("Request %d served. Head movement: %d\n", req[i], hm);
```

```

        total = total + hm;
        head = req[i];
    }

    printf("\nTotal Head Movement=%d",total);
}

void main()
{
    int req[20],i,n,head;

    printf("Enter the no.of processes to be requested : ");
    scanf("%d",&n);

    printf("Enter the request : ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);

    sort(req,n);

    printf("\nSorted Request : ");
    for(i=0;i<n;i++)
        printf("\t%d",req[i]);

    printf("\nEnter the initial head position : ");
    scanf("%d",&head);

    printf("\nSeek Sequence : %d",head);

    look(req,n,head);
}

```

Q17 C-LOOK

Requests: 23, 89, 132, 42, 187, 69, 36, 55.Start Head Position: 40.Direction: Right

Q31 C-LOOK

Requests: 80, 150, 60, 135, 40, 35, 170.Start Head Position: 70.Direction: Right

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
void sort(int req[], int n)
```

```
{
```

```
    int i,j,temp;
```

```
    for(i=0;i<n-1;i++)
```

```
    {
```

```
        for(j=i+1;j<n;j++)
```

```
        {
```

```
            if(req[i]>req[j])
```

```
            {
```

```
                temp=req[i];
```

```
                req[i]=req[j];
```

```
                req[j]=temp;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
void clook(int req[],int n,int head)
```

```
{
```

```
    int i,index,hm,total=0;
```

```
    printf("Direction: Right\n");
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        if(req[i]>head)
```

```
        {
```

```

        index=i;
        break;
    }
}
for(i=index;i<n;i++)
{
    hm=abs(req[i]-head);
    printf("Request %d served. Head movement: %d\n",req[i],hm);
    total=total+hm;
    head=req[i];
}
printf("Jump to beginning of request queue\n");
hm=abs(head-req[0]);
total=total+hm;
head=req[0];
for(i=0;i<index;i++)
{
    hm=abs(req[i]-head);
    printf("Request %d served. Head movement: %d\n",req[i],hm);
    total=total+hm;
    head=req[i];
}
printf("\nTotal Head Movement=%d",total);
}
void main()
{
    int req[20],i,n,head;
    printf("Enter the no.of processes to be requested : ");
    scanf("%d",&n);
    printf("Enter the request : ");

```

```

for(i=0;i<n;i++)
    scanf("%d",&req[i]);
sort(req,n);
printf("\nSorted Request : ");
for(i=0;i<n;i++)
    printf("\t%d",req[i]);
printf("\nEnter the initial head position : ");
scanf("%d",&head);
printf("\nSeek Sequence : %d",head);
clook(req,n,head);
}

```

Q30 C-LOOK

Requests: 56, 59, 40, 19, 91, 161, 151, 39, 185.Start Head Position: 48.Direction: User Defined

```

#include<stdio.h>
#include<stdlib.h>
void sort(int req[], int n)
{
    int i,j,temp;
    for(i=0;i<n-1;i++)
    {
        for(j=i+1;j<n;j++)
        {
            if(req[i]>req[j])
            {
                temp=req[i];
                req[i]=req[j];
                req[j]=temp;
            }
        }
    }
}

```

```

    }
}
void clook(int req[],int n,int head,int dir)
{
    int i,index,hm,total=0;
    if(dir==1)
        printf("Direction: Right\n");
    else
        printf("Direction: Left\n");
    for(i=0;i<n;i++)
    {
        if(req[i]>head)
        {
            index=i;
            break;
        }
    }
    if(dir==1)
    {
        for(i=index;i<n;i++)
        {
            hm=abs(req[i]-head);
            printf("Request %d served. Head movement: %d\n",req[i],hm);
            total+=hm;
            head=req[i];
        }
        printf("Jump to beginning of request queue\n");
        hm=abs(head-req[0]);
        total+=hm;
        head=req[0];
    }
}

```

```

for(i=0;i<index;i++)
{
    hm=abs(req[i]-head);
    printf("Request %d served. Head movement: %d\n",req[i],hm);
    total+=hm;
    head=req[i];
}
}
else
{
    for(i=index-1;i>=0;i--)
    {
        hm=abs(req[i]-head);
        printf("Request %d served. Head movement: %d\n",req[i],hm);
        total+=hm;
        head=req[i];
    }
    printf("Jump to end of request queue\n");
    hm=abs(head-req[n-1]);
    total+=hm;
    head=req[n-1];
    for(i=n-1;i>=index;i--)
    {
        hm=abs(req[i]-head);
        printf("Request %d served. Head movement: %d\n",req[i],hm);
        total+=hm;
        head=req[i];
    }
}
printf("\nTotal Head Movement=%d",total);

```

```

}

void main()
{
    int req[20],i,n,head,dir;
    printf("Enter the no.of processes to be requested : ");
    scanf("%d",&n);
    printf("Enter the request : ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);
    sort(req,n);
    printf("\nSorted Request : ");
    for(i=0;i<n;i++)
        printf("\t%d",req[i]);
    printf("\nEnter the initial head position : ");
    scanf("%d",&head);
    printf("\nEnter Direction (1=Right, 0=Left): ");
    scanf("%d",&dir);
    printf("\nSeek Sequence : %d\n",head);
    clook(req,n,head,dir);
}

```

Compile

mpicc program.c

Run

mpirun -np 4 ./a.out

Q4.MPI Program – Sum of 1000 Random Numbers

```
#include<stdio.h>
#include<stdlib.h>
#include<mpi.h>
int main(int argc,char *argv[])
{
    int rank,size,i;
    int numbers[1000];
    int local_sum=0,total_sum=0;
    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    MPI_Comm_size(MPI_COMM_WORLD,&size);
    if(rank==0)
    {
        for(i=0;i<1000;i++)
            numbers[i]=rand()%100;
    }
    MPI_Bcast(numbers,1000,MPI_INT,0,MPI_COMM_WORLD);
    for(i=rank;i<1000;i+=size)
        local_sum+=numbers[i];
    MPI_Reduce(&local_sum,&total_sum,1,MPI_INT,MPI_SUM,0,MPI_COMM_WORLD);
    if(rank==0)
        printf("Total Sum = %d\n",total_sum);

    MPI_Finalize();
}
```

Q16.MPI Program – Sum and Average

```
#include<stdio.h>
#include<stdlib.h>
#include<mpi.h>
int main(int argc,char *argv[])
{
    int rank,size,i;
    int numbers[1000];
    int local_sum=0,total_sum=0;
    float avg;
```

```

MPI_Init(&argc,&argv);
MPI_Comm_rank(MPI_COMM_WORLD,&rank);
MPI_Comm_size(MPI_COMM_WORLD,&size);
if(rank==0)
{
    for(i=0;i<1000;i++)
        numbers[i]=rand()%100;
}
MPI_Bcast(numbers,1000,MPI_INT,0,MPI_COMM_WORLD);
for(i=rank;i<1000;i+=size)
    local_sum+=numbers[i];
MPI_Reduce(&local_sum,&total_sum,1,MPI_INT,MPI_SUM,0,MPI_COMM_WORLD);
if(rank==0)
{
    avg=(float)total_sum/1000;
    printf("Sum = %d\n",total_sum);
    printf("Average = %f\n",avg);
}

MPI_Finalize();
}

```

Q27. Write an MPI program to calculate sum of all odd randomly generated 1000 numbers(stored in array) on a cluster.

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
#define SIZE 1000
int main(int argc, char *argv[])
{
    int rank, size;
    int numbers[SIZE];
    int local_sum = 0, global_sum = 0;
    int i;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if(rank == 0)

```

```

{
    for(i = 0; i < SIZE; i++)
    {
        numbers[i] = rand() % 100;
    }
}

MPI_Bcast(numbers, SIZE, MPI_INT, 0, MPI_COMM_WORLD);

for(i = rank; i < SIZE; i += size)
{
    if(numbers[i] % 2 != 0)
    {
        local_sum += numbers[i];
    }
}

MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);

if(rank == 0)
{
    printf("Sum of all odd numbers = %d\n", global_sum);
}

MPI_Finalize();

return 0;
}

```

Q33.MPI Program – Minimum Number

```

#include<stdio.h>
#include<stdlib.h>
#include<mpi.h>
int main(int argc,char *argv[])
{
    int rank,size,i;
    int numbers[1000];
    int local_min=9999,global_min;
    MPI_Init(&argc,&argv);

```

```

MPI_Comm_rank(MPI_COMM_WORLD,&rank);
MPI_Comm_size(MPI_COMM_WORLD,&size);

if(rank==0)
{
    for(i=0;i<1000;i++)
        numbers[i]=rand()%1000;
}

MPI_Bcast(numbers,1000,MPI_INT,0,MPI_COMM_WORLD);
for(i=rank;i<1000;i+=size)
    if(numbers[i]<local_min)
        local_min=numbers[i];

MPI_Reduce(&local_min,&global_min,1,MPI_INT,MPI_MIN,0,MPI_COMM_WORLD);
if(rank==0)
    printf("Minimum Number = %d\n",global_min);

MPI_Finalize();
}

```

Q34. MPI Program – Maximum Number

```

#include<stdio.h>
#include<stdlib.h>
#include<mpi.h>
int main(int argc,char *argv[])
{
    int rank,size,i;
    int numbers[1000];
    int local_max=0,global_max;
    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    MPI_Comm_size(MPI_COMM_WORLD,&size);
    if(rank==0)
    {
        for(i=0;i<1000;i++)
            numbers[i]=rand()%1000;
    }

    MPI_Bcast(numbers,1000,MPI_INT,0,MPI_COMM_WORLD);
    for(i=rank;i<1000;i+=size)
        if(numbers[i]>local_max)
            local_max=numbers[i];
}

```

```

MPI_Reduce(&local_max,&global_max,1,MPI_INT,MPI_MAX,0,MPI_COMM_WORLD);

if(rank==0)
    printf("Maximum Number = %d\n",global_max);

MPI_Finalize();
}

```

Q35.MPI Program – Root Process Generates Random Array

```

#include<stdio.h>
#include<stdlib.h>
#include<mpi.h>
int main(int argc,char *argv[])
{
    int rank,i;
    int numbers[10];

    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);

    if(rank==0)
    {
        printf("Root generating numbers:\n");
        for(i=0;i<10;i++)
        {
            numbers[i]=rand()%100;
            printf("%d ",numbers[i]);
        }
    }

    MPI_Bcast(numbers,10,MPI_INT,0,MPI_COMM_WORLD);

    if(rank!=0)
    {
        printf("\nProcess %d received numbers: ",rank);
        for(i=0;i<10;i++)
            printf("%d ",numbers[i]);
    }

    MPI_Finalize();
}

```

Q 1. Write a C Menu driven Program to implement following functionality a) Accept Available

b) Display Allocation, Max c) Display the contents of need matrix d) Display Available

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	2	3	2	9	7	5	3	3	2
P1	4	0	0	5	2	2			
P2	5	0	4	1	0	4			
P3	4	3	3	4	4	4			
P4	2	2	4	6	5	5			

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int allocation[5][3] = {
```

```
        {2,3,2},
```

```
        {4,0,0},
```

```
        {5,0,4},
```

```
        {4,3,3},
```

```
        {2,2,4}
```

```
    };
```

```
    int max[5][3] = {
```

```
        {9,7,5},
```

```
        {5,2,2},
```

```
        {1,0,4},
```

```
        {4,4,4},
```

```
        {6,5,5}
```

```

};
int need[5][3];
int available[3];
int i,j,choice;
do
{
    printf("\n--- MENU ---\n");
    printf("1. Accept Available\n");
    printf("2. Display Allocation and Max\n");
    printf("3. Display Need Matrix\n");
    printf("4. Display Available\n");
    printf("5. Exit\n");
    printf("Enter your choice: ");
    scanf("%d",&choice);
    switch(choice)
    {
    case 1:
        printf("Enter Available Resources (A B C): ");
        for(i=0;i<3;i++)
            scanf("%d",&available[i]);
        break;
    case 2:
        printf("\nAllocation Matrix\n");
        for(i=0;i<5;i++)
        {
            for(j=0;j<3;j++)
                printf("%d ",allocation[i][j]);
            printf("\n");
        }
        printf("\nMax Matrix\n");

```

```

        for(i=0;i<5;i++)
        {
            for(j=0;j<3;j++)
                printf("%d ",max[i][j]);
            printf("\n");
        }
break;
case 3:
    printf("\nNeed Matrix\n");
    for(i=0;i<5;i++)
    {
        for(j=0;j<3;j++)
        {
            need[i][j] = max[i][j] - allocation[i][j];
            printf("%d ",need[i][j]);
        }
        printf("\n");
    }
break;
case 4:
    printf("\nAvailable Resources: ");
    for(i=0;i<3;i++)
        printf("%d ",available[i]);
break;

    }
}while(choice!=5);
}

```

Q5. Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. Consider the following snapshot of system A, B, C and D is the resource type.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

a) Calculate and display the content of need matrix? b) Is the system in safe state? If display the safe sequence.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int allocation[5][4] = {
```

```
        {0,0,1,2},
```

```
        {1,0,0,0},
```

```
        {1,3,5,4},
```

```
        {0,6,3,2},
```

```
        {0,0,1,4}
```

```
    };
```

```
    int max[5][4] = {
```

```
        {0,0,1,2},
```

```
        {1,7,5,0},
```

```
        {2,3,5,6},
```

```

        {0,6,5,2},
        {0,6,5,6}
    };
    int available[4] = {1,5,2,0};
    int need[5][4];
    int finish[5]={0};
    int safeSeq[5];
    int work[4];
    int i,j,k,count=0,flag;
    printf("Need Matrix:\n");
    for(i=0;i<5;i++)
    {
        for(j=0;j<4;j++)
        {
            need[i][j] = max[i][j] - allocation[i][j];
            printf("%d ",need[i][j]);
        }
        printf("\n");
    }
    for(i=0;i<4;i++)
        work[i]=available[i];
    while(count<5)
    {
        flag=0;
        for(i=0;i<5;i++)
        {
            if(finish[i]==0)
            {
                int possible=1;

```

```

    for(j=0;j<4;j++)
    {
        if(need[i][j] > work[j])
        {
            possible=0;
            break;
        }
    }
    if(possible)
    {
        for(k=0;k<4;k++)
            work[k]+=allocation[i][k];
        safeSeq[count++]=i;
        finish[i]=1;
        flag=1;
    }
}
if(flag==0)
    break;
}
if(count==5)
{
    printf("\nSystem is in SAFE state\n");
    printf("Safe Sequence: ");
    for(i=0;i<5;i++)
        printf("P%d ",safeSeq[i]);
}
else
{

```

```

printf("\nSystem is NOT in safe state");

}

}

```

Q6.Implement the Menu driven Banker's algorithm for accepting Allocation Max from user.

- a) Accept Available**
- b)Display Allocation, Max**
- c) Find Need and display It,**
- d)Display Available**

Consider the system with 3 resources types A,B and C with 7,2,6 instances respectively. Consider the following snapshot:

Process	Allocation				Request		
	A	B	C		A	B	C
P0	0	1	0		0	0	0
P1	4	0	0		5	2	2
P2	5	0	4		1	0	4
P3	4	3	3		4	4	4
P4	2	2	4		6	5	5

```

#include<stdio.h>

int main()
{
    int allocation[5][3]={
        {0,1,0},
        {4,0,0},

```

```

        {5,0,4},
        {4,3,3},
        {2,2,4}
};

int max[5][3]={
    {0,0,0},
    {5,2,2},
    {1,0,4},
    {4,4,4},
    {6,5,5}
};

int need[5][3];
int available[3];
int i,j,choice;
do
{
    printf("\n--- MENU ---\n");
    printf("1. Accept Available\n");
    printf("2. Display Allocation and Max\n");
    printf("3. Find and Display Need Matrix\n");
    printf("4. Display Available\n");
    printf("5. Exit\n");
    printf("Enter choice: ");
    scanf("%d",&choice);
    switch(choice)
    {
        case 1:
            printf("Enter Available Resources (A B C): ");
            for(i=0;i<3;i++)
                scanf("%d",&available[i]);

```

break;

case 2:

```
printf("\nAllocation Matrix\n");
for(i=0;i<5;i++)
{
    for(j=0;j<3;j++)
        printf("%d ",allocation[i][j]);
    printf("\n");
}
printf("\nMax Matrix\n");
for(i=0;i<5;i++)
{
    for(j=0;j<3;j++)
        printf("%d ",max[i][j]);
    printf("\n");
}
```

break;

case 3:

```
printf("\nNeed Matrix\n");
for(i=0;i<5;i++)
{
    for(j=0;j<3;j++)
    {
        need[i][j] = max[i][j] - allocation[i][j];
        printf("%d ",need[i][j]);
    }
    printf("\n");
}
```

break;

case 4:

```

printf("\nAvailable Resources: ");

for(i=0;i<3;i++)

printf("%d ",available[i]);

break;

}

}while(choice!=5);

}

```

Q10.Consider the following snapshot of the system.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	0	0	1	4	2	1	2	3	3	2	1
P1	3	1	2	1	5	2	5	2				
P2	2	1	0	3	2	3	1	6				
P3	1	3	1	2	1	4	2	4				
P4	1	4	3	2	3	6	6	5				

Using Resource –Request algorithm to Check whether the current system is in safe state or not .same as Q5 only matrix value changes

```

#include<stdio.h>

int main()

{

int allocation[5][4] = {

{2,0,0,1},

{3,1,2,1},

{2,1,0,3},

{1,3,1,2},

```

```

        {1,4,3,2}
    };
    int max[5][4] = {
        {4,2,1,2},
        {5,2,5,2},
        {2,3,1,6},
        {1,4,2,4},
        {3,6,6,5}
    };
    int available[4] = {3,3,2,1};
    int need[5][4];
    int finish[5] = {0};
    int safeSeq[5];
    int work[4];
    int i,j,k,count=0;
    printf("Need Matrix:\n");
    for(i=0;i<5;i++)
    {
        for(j=0;j<4;j++)
        {
            need[i][j] = max[i][j] - allocation[i][j];
            printf("%d ",need[i][j]);
        }
        printf("\n");
    }
    for(i=0;i<4;i++)
        work[i] = available[i];
    while(count < 5)
    {
        int found = 0;

```

```

for(i=0;i<5;i++)
{
    if(finish[i] == 0)
    {
        int possible = 1;
        for(j=0;j<4;j++)
        {
            if(need[i][j] > work[j])
            {
                possible = 0;
                break;
            }
        }
        if(possible)
        {
            for(k=0;k<4;k++)
                work[k] += allocation[i][k];
            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
        }
    }
}
if(found == 0)
    break;
}
if(count == 5)
{
    printf("\nSystem is in SAFE STATE\nSafe Sequence: ");
    for(i=0;i<5;i++)

```

```

        printf("P%d ", safeSeq[i]);
    }
    else
    {
        printf("\nSystem is NOT in safe state");
    }
}

```

12. Consider the following snapshot of system, A, B, C, D is the resource type.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

Using Resource –Request algorithm to Check whether the current system is in safe state or not . Same as Q5 only matrix value changes

```

#include<stdio.h>

int main()
{
    int allocation[5][4] = {
        {0,0,1,2},
        {1,0,0,0},
        {1,3,5,4},
        {0,6,3,2},

```

```

{0,0,1,4}
};
int max[5][4] = {
{0,0,1,2},
{1,7,5,0},
{2,3,5,6},
{0,6,5,2},
{0,6,5,6}
};
int available[4] = {1,5,2,0};
int need[5][4];
int finish[5]={0};
int safe[5];
int work[4];
int i,j,k,count=0,flag;
printf("Need Matrix:\n");
for(i=0;i<5;i++)
{
for(j=0;j<4;j++)
{
need[i][j] = max[i][j] - allocation[i][j];
printf("%d ",need[i][j]);
}
printf("\n");
}
for(i=0;i<4;i++)
work[i]=available[i];
while(count<5)
{
flag=0;

```

```
for(i=0;i<5;i++)
{
if(finish[i]==0)
{
int possible=1;
for(j=0;j<4;j++)
{
if(need[i][j] > work[j])
{
possible=0;
break;
}
}
if(possible)
{
for(k=0;k<4;k++)
work[k]+=allocation[i][k];
safe[count++]=i;
finish[i]=1;
flag=1;
}
}
if(flag==0)
break;
}
if(count==5)
{
printf("\nSystem is in Safe State\nSafe Sequence: ");
for(i=0;i<5;i++)
```

```

printf("P%d ",safe[i]);
}
else
printf("\nSystem is NOT in Safe State");
}

```

Q15. Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. the following snapshot of system, A, B, C and D are the resource type.

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			

P2	3	0	3	0	0	0			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

Implement the following Menu. a) Accept Available b) Display Allocation, Max c) Display the contents of need matrix d) Display Available

```

#include<stdio.h>

int allocation[5][3] = {
{0,1,0},
{2,0,0},
{3,0,3},
{2,1,1},
{0,0,2}

```

```

};

int max[5][3] = {
    {0,0,0},
    {2,0,2},
    {0,0,0},
    {1,0,0},
    {0,0,2}
};

int available[3];
int need[5][3];

void acceptAvailable()
{
    int i;
    printf("Enter Available Resources (A B C): ");
    for(i=0;i<3;i++)
        scanf("%d",&available[i]);
}

void displayAllocationMax()
{
    int i,j;
    printf("\nProcess Allocation    Max\n");
    for(i=0;i<5;i++)
    {
        printf("P%d    ",i);
        for(j=0;j<3;j++)
            printf("%d ",allocation[i][j]);
        printf("    ");
        for(j=0;j<3;j++)
            printf("%d ",max[i][j]);
        printf("\n");
    }
}

```

```

    }
}
void displayNeed()
{
    int i,j;
    printf("\nNeed Matrix\n");
    for(i=0;i<5;i++)
    {
        printf("P%d ",i);
        for(j=0;j<3;j++)
        {
            need[i][j]=max[i][j]-allocation[i][j];
            printf("%d ",need[i][j]);
        }
        printf("\n");
    }
}
void displayAvailable()
{
    printf("\nAvailable Resources: ");
    printf("A=%d B=%d C=%d\n",available[0],available[1],available[2]);
}
int main()
{
    int ch;
    do
    {
        printf("\n----- MENU ----- \n");
        printf("1.Accept Available\n");
        printf("2.Display Allocation and Max\n");
    }
}

```

```

printf("3.Display Need Matrix\n");
printf("4.Display Available\n");
printf("5.Exit\n");
printf("Enter choice: ");
scanf("%d",&ch);
switch(ch)
{
    case 1: acceptAvailable();
            break;
    case 2: displayAllocationMax();
            break;
    case 3: displayNeed();
            break;
    case 4: displayAvailable();
            break;
    case 5: break;
    default: printf("Invalid Choice");
}
}while(ch!=5);
}

```

Q18. Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. the following snapshot of system, A, B, C and D are the resource type.

Processes	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			

P2	3	0	3	0	0	0			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

a) Calculate and display the content of need matrix? b) Is the system in safe state? If display the safe sequence.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int allocation[5][3] = {
```

```
        {0,1,0},
```

```
        {2,0,0},
```

```
        {3,0,3},
```

```
        {2,1,1},
```

```
        {0,0,2}
```

```
    };
```

```
    int max[5][3] = {
```

```
        {0,0,0},
```

```
        {2,0,2},
```

```
        {0,0,0},
```

```
        {1,0,0},
```

```
        {0,0,2}
```

```
    };
```

```
    int available[3] = {0,0,0};
```

```
    int need[5][3];
```

```
    int finish[5]={0};
```

```
    int safe[5];
```

```
    int work[3];
```

```
    int i,j,k,count=0;
```

```

printf("Need Matrix:\n");
for(i=0;i<5;i++)
{
    for(j=0;j<3;j++)
    {
        need[i][j] = max[i][j] - allocation[i][j];
        printf("%d ",need[i][j]);
    }
    printf("\n");
}
for(i=0;i<3;i++)
    work[i]=available[i];
while(count<5)
{
    int found=0;
    for(i=0;i<5;i++)
    {
        if(finish[i]==0)
        {
            int possible=1;
            for(j=0;j<3;j++)
            {
                if(need[i][j] > work[j])
                {
                    possible=0;
                    break;
                }
            }
            if(possible)
            {

```

```

        for(k=0;k<3;k++)
            work[k]+=allocation[i][k];
        safe[count++]=i;
        finish[i]=1;
        found=1;
    }
}
}
if(found==0)
    break;
}
if(count==5)
{
    printf("\nSystem is in SAFE STATE\nSafe Sequence: ");
    for(i=0;i<5;i++)
        printf("P%d ",safe[i]);
}
else
    printf("\nSystem is NOT in Safe State");
}

```

Q24. Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. Consider the following snapshot of system, A,B,C & D is the resource type.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	3	2	4	6	5	4	4	3	4	4	2
P1	1	2	0	1	4	4	4	4				

P2	0	0	0	0	0	0	1	2				
P3	3	3	2	2	3	9	3	4				
P4	1	4	3	2	2	5	3	3				
P5	2	4	1	4	4	6	3	4				

a) Calculate and display the content of need matrix? b) Is the system in safe state If display the safe sequence.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int allocation[6][4] = {
```

```
{0,3,2,4},
```

```
{1,2,0,1},
```

```
{0,0,0,0},
```

```
{3,3,2,2},
```

```
{1,4,3,2},
```

```
{2,4,1,4}
```

```
};
```

```
int max[6][4] = {
```

```
{6,5,4,4},
```

```
{4,4,4,4},
```

```
{0,0,1,2},
```

```
{3,9,3,4},
```

```
{2,5,3,3},
```

```
{4,6,3,4}
```

```
};
```

```
int available[4] = {3,4,4,2};
```

```
int need[6][4];
```

```
int finish[6]={0};
```

```

int safe[6];
int work[4];
int i,j,k,count=0;
printf("Need Matrix:\n");
for(i=0;i<6;i++)
{
for(j=0;j<4;j++)
{
need[i][j] = max[i][j] - allocation[i][j];
printf("%d ",need[i][j]);
}
printf("\n");
}
for(i=0;i<4;i++)
work[i] = available[i];
while(count < 6)
{
int found = 0;
for(i=0;i<6;i++)
{
if(finish[i] == 0)
{
int possible = 1;
for(j=0;j<4;j++)
{
if(need[i][j] > work[j])
{
possible = 0;
break;
}
}
}
}
}

```

```

}
if(possible)
{
for(k=0;k<4;k++)
work[k] += allocation[i][k];
safe[count++] = i;
finish[i] = 1;
found = 1;
}
}
}
if(found == 0)
break;
}
if(count == 6)
{
printf("\nSystem is in SAFE STATE\nSafe Sequence: ");
for(i=0;i<6;i++)
printf("P%d ",safe[i]);
}
else
printf("\nSystem is NOT in Safe State");
}

```

Q8.Consider a system with ‘m’ processes and ‘n’ resource types. Accept number of instances for every resource type. For each process accept the allocation and maximum requirement matrices. Write a program to display the contents of need matrix and to check if the given request of a process can be granted immediately or not

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```

int i,j,m,n,p;
printf("Enter number of processes: ");
scanf("%d",&m);
printf("Enter number of resource types: ");
scanf("%d",&n);
int alloc[m][n], max[m][n], need[m][n];
int avail[n], request[n];
printf("\nEnter Allocation Matrix:\n");
for(i=0;i<m;i++)
for(j=0;j<n;j++)
scanf("%d",&alloc[i][j]);
printf("\nEnter Max Matrix:\n");
for(i=0;i<m;i++)
for(j=0;j<n;j++)
scanf("%d",&max[i][j]);
printf("\nEnter Available Resources:\n");
for(i=0;i<n;i++)
scanf("%d",&avail[i]);
printf("\nEnter Need Matrix:\n");
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
need[i][j]=max[i][j]-alloc[i][j];
printf("%d ",need[i][j]);
}
printf("\n");
}
printf("\nEnter process number making request: ");
scanf("%d",&p);

```

```
printf("Enter request for resources:\n");
for(i=0;i<n;i++)
scanf("%d",&request[i]);
int flag=1;
for(i=0;i<n;i++)
{
if(request[i] > need[p][i])
{
printf("\nError: Request exceeds Need");
flag=0;
break;
}
}
if(flag)
{
for(i=0;i<n;i++)
{
if(request[i] > avail[i])
{
printf("\nResources not available. Process must wait.");
flag=0;
break;
}
}
}
if(flag)
printf("\nRequest can be granted immediately.");
}
```

Q20. Write a program to simulate Sequential (Contiguous) file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option.

· Show Bit Vector · Show Directory · Delete File · Exit

```
#include<stdio.h>

#include<stdlib.h>

int main()
{
    int n,i,j,ch,f,start[20],length[20],b[50];
    printf("Enter number of disk blocks: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
        b[i]=0;
    for(i=0;i<n/4;i++)
    {
        int k=rand()%n;
        b[k]=1;
    }
    do
    {
        printf("\n1.Show Bit Vector");
        printf("\n2.Show Directory");
        printf("\n3.Delete File");
        printf("\n4.Exit");
        printf("\nEnter choice: ");
        scanf("%d",&ch);
```

```

switch(ch)
{
case 1:
    printf("\nBit Vector:\n");
    for(i=0;i<n;i++)
        printf("%d ",b[i]);
    break;
case 2:
    printf("\nEnter number of files: ");
    scanf("%d",&f);
    for(i=0;i<f;i++)
    {
        printf("\nEnter starting block for file %d: ",i);
        scanf("%d",&start[i]);
        printf("Enter length of file: ");
        scanf("%d",&length[i]);
        for(j=start[i]; j<start[i]+length[i]; j++)
        {
            if(b[j]==0)
                b[j]=1;
            else
            {
                printf("Block already allocated\n");
                break;
            }
        }
    }
    printf("\nDirectory\n");
    for(i=0;i<f;i++)
        printf("\nFile %d -> Start:%d Length:%d",i,start[i],length[i]);

```

```

        break;
    case 3:
        printf("\nEnter file number to delete: ");
        scanf("%d",&i);
        for(j=start[i]; j<start[i]+length[i]; j++)
            b[j]=0;
        printf("File Deleted");
        break;
    case 4:
        break;
    default:
        printf("Invalid Choice");
    }
}while(ch!=4);
}

```

Q21. Write a program to simulate Indexed file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned above and implement each option.

· Show Bit Vector · Show Directory · Delete Already File · Exit

```

#include<stdio.h>
#include<stdlib.h>

int main()
{
    int n,i,j,k,ch,f,b[50],fno=0;
    int index[20],file[20][20],count[20];
    printf("Enter number of disk blocks: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
        b[i]=0;

```

```

for(i=0;i<n/4;i++)
{
    k=rand()%n;
    b[k]=1;
}
do
{
    printf("\n1.Show Bit Vector");
    printf("\n2.Show Directory");
    printf("\n3.Delete File");
    printf("\n4.Exit");
    printf("\nEnter choice: ");
    scanf("%d",&ch);
    switch(ch)
    {
    case 1:
        printf("\nBit Vector:\n");
        for(i=0;i<n;i++)
            printf("%d ",b[i]);
        break;
    case 2:
        printf("\nEnter number of files: ");
        scanf("%d",&f);
        for(i=0;i<f;i++)
        {
            printf("\nEnter index block for file %d: ",i);
            scanf("%d",&index[i]);
            if(b[index[i]]==0)
            {
                b[index[i]]=1;
            }
        }
    }
}

```

```

        printf("Enter number of blocks: ");
        scanf("%d",&count[i]);
        printf("Enter blocks: ");
        for(j=0;j<count[i];j++) {
            scanf("%d",&file[i][j]);
            b[file[i][j]]=1;
        }
    }
    else
        printf("Block already allocated");
}

printf("\nDirectory\n");
for(i=0;i<f;i++)
{
    printf("\nFile %d -> Index %d -> ",i,index[i]);
    for(j=0;j<count[i];j++)
        printf("%d ",file[i][j]);
}

break;
case 3:
    printf("\nEnter file number to delete: ");
    scanf("%d",&i);
    b[index[i]]=0;
    for(j=0;j<count[i];j++)
        b[file[i][j]]=0;
    printf("File Deleted");

break;
case 4:
break;
default:

```

```

        printf("Invalid choice");
    }
}while(ch!=4);
}

```

Q3. Write a program to simulate Linked file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option.

• Show Bit Vector • Create New File • Show Directory • Exit

```

#include<stdio.h>
#include<stdlib.h>

int disk[100], n;

struct file
{
    char name[20];
    int start;
    int blocks[20];
    int count;
}f[20];

int filecount=0;

void showbitvector()
{
    int i;
    printf("\nBit Vector:\n");
    for(i=0;i<n;i++)
        printf("%d ",disk[i]);
}

void createfile()
{
    int i,b;

```

```

printf("\nEnter file name: ");
scanf("%s",f[filecount].name);
printf("Enter number of blocks required: ");
scanf("%d",&b);
f[filecount].count=b;
printf("Enter block numbers:\n");
for(i=0;i<b;i++)
{
    scanf("%d",&f[filecount].blocks[i]);

    if(disk[f[filecount].blocks[i]]==0)
    {
        disk[f[filecount].blocks[i]]=1;
    }
    else
    {
        printf("Block already allocated\n");
        i--;
    }
}
f[filecount].start=f[filecount].blocks[0];
filecount++;
}

```

```

void showdirectory()
{
    int i,j;
    printf("\nDirectory:\n");
    for(i=0;i<filecount;i++)
    {

```

```

        printf("\nFile Name: %s",f[i].name);
        printf("\nStart Block: %d",f[i].start);
        printf("\nBlocks: ");

        for(j=0;j<f[i].count;j++)
            printf("%d ",f[i].blocks[j]);

        printf("\n");
    }
}

int main()
{
    int i,choice;
    printf("Enter number of disk blocks: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
        disk[i]=rand()%2;
    while(1)
    {
        printf("\n\n1.Show Bit Vector");
        printf("\n2.Create New File");
        printf("\n3.Show Directory");
        printf("\n4.Exit");
        printf("\nEnter choice: ");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1: showbitvector();
                    break;
            case 2: createfile();

```

```
        break;
    case 3: showdirectory();
        break;
    case 4: exit(0);
    default: printf("Invalid choice");
}
}
return 0;
}
```

Data Analytics

Linear Regression

1. Create 'Position Salaries' Data set. Build a linear regression model by identifying independent and target variable. Split the variables into training and testing sets. then divide the training and testing sets into a 7:3 ratio, respectively and print them. Build a simple linear regression model.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
np.random.seed(42)
df = pd.DataFrame({
    "Position_Level": np.arange(1, 11),
    "Salary":
    [45000,50000,60000,80000,110000,150000,200000,300000,500000,1000000]
})
X = df[["Position_Level"]]
y = df["Salary"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
print(df.head())
print(X_train.head())
print(X_test.head())
print(y_train.head())
```

```
print(y_test.head())
model = LinearRegression()
model.fit(X_train, y_train)
print(model.intercept_)
print(model.coef_)
print(model.predict(X_test))
```

2.Create Salary Data set. Build a linear regression model by identifying independent and target variable. Split the variables into training and testing sets and print them. Build a simple linear regression model for predicting purchases.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
np.random.seed(42)
df = pd.DataFrame({
    "ID": np.arange(1, 501),
    "Age": np.random.randint(20, 60, 500),
    "Experience": np.random.randint(1, 30, 500),
    "Salary": np.random.uniform(20000, 120000, 500),
    "Purchases": np.random.randint(0, 2, 500)
})
X = df[["Age", "Experience", "Salary"]]
y = df["Purchases"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
print(df.head())
print(X_train.head())
print(X_test.head())
print(y_train.head())
print(y_test.head())
model = LinearRegression()
model.fit(X_train, y_train)
print(model.intercept_)
print(model.coef_)
print(model.predict(X_test)[:10])
```

3. Create 'User' Data set having 5 columns namely: User ID, Gender, Age, Estimated Salary and Purchased. Build a linear regression model that can predict whether on the given parameter a person will buy a car or not.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import LabelEncoder
np.random.seed(42)
df = pd.DataFrame({
    "UserID": np.arange(1,501),
    "Gender": np.random.choice(["Male","Female"],500),
    "Age": np.random.randint(18,65,500),
    "EstimatedSalary": np.random.randint(20000,150000,500),
    "Purchased": np.random.choice([0,1],500)
})
```

```

print("Dataset:")
print(df.head())
le = LabelEncoder()
df["Gender"] = le.fit_transform(df["Gender"])
X = df[["Gender", "Age", "EstimatedSalary"]]
y = df["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
print("Model Score:", model.score(X_test, y_test))
pred = model.predict(X_test)
print("First 10 Predictions:", pred[:10])

```

4. Build a simple linear regression model for Fish Species Weight Prediction. (download dataset <https://www.kaggle.com/aungpyaeap/fish-market?select=Fish.csv>)

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
df = pd.read_csv("Fish.csv")
df = df.dropna()
X = df[["Length1", "Length2", "Length3", "Height", "Width"]]
y = df["Weight"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42

```

```

)
print(df.head())
print(X_train.head())
print(X_test.head())
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
print("Predictions:", y_pred[:10])

```

5. Use the iris dataset. Write a Python program to view some basic statistical details like percentile, mean, std etc. of the species of 'Iris-setosa', 'Iris-versicolor' and 'Iris-virginica'. Apply linear regression on the dataset to identify different species (setosa, versicolor, virginica) of Iris flowers given just 4 features: sepal and petal lengths and widths. Find the accuracy of the model.

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
df = pd.read_csv("iris.csv")
print("Dataset:")
print(df.head())
stats = df.groupby("species").describe()
print("\nStatistical Details:")

```

```

print(stats)
le = LabelEncoder()
df["species"] = le.fit_transform(df["species"])
X = df.drop("species", axis=1)
y = df["species"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_pred_class = [round(i) for i in y_pred]
accuracy = accuracy_score(y_test, y_pred_class)
print("\nAccuracy:", accuracy)
print("First 10 Predictions:", y_pred_class[:10])

```

6. Create ‘heights-and-weights’ Data set. Build a linear regression model by identifying independent and target variable. Split the variables into training and testing sets and print them. Build a simple linear regression model for predicting purchases.

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
np.random.seed(42)
df = pd.DataFrame({
    "Height": np.random.randint(150, 190, 100), # independent variable
    "Weight": np.random.randint(50, 100, 100) # target variable

```

```

}))
print("Dataset:")
print(df.head())
X = df[["Height"]]
y = df["Weight"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
print("\nX_train:")
print(X_train.head())
print("\nX_test:")
print(X_test.head())
print("\ny_train:")
print(y_train.head())
print("\ny_test:")
print(y_test.head())
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("\nFirst 10 Predictions:")
print(y_pred[:10])

```

Apriori Algorithm:

1./4. Create the following dataset in python & convert the categorical values into numeric format. Apply the apriori algorithm on the above dataset to generate the frequent itemsets and association rules. Repeat the process with different min_sup values.

TID	Items
1	'eggs', 'milk','bread'
2	'eggs', 'apple'
3	'milk', 'bread'
4	'apple', 'milk'
5	'milk', 'apple', 'bread'

```

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
transactions = [
    ['eggs', 'milk','bread'],
    ['eggs', 'apple'],
    ['milk', 'bread'],
    ['apple', 'milk'],
    ['milk', 'apple', 'bread']
]

from mlxtend.preprocessing import TransactionEncoder
te=TransactionEncoder()
te_array=te.fit(transactions).transform(transactions)
df=pd.DataFrame(te_array, columns=te.columns_)
df
print(te_array)
print(te.columns_)
freq_items = apriori(df, min_support = 0.5, use_colnames = True)
print(freq_items)
rules = association_rules(freq_items, metric ='support', min_threshold=0.05 )
rules = rules.sort_values(['support', 'confidence'], ascending =[False,False])

```

```
print(rules)
```

2. Write a python program to read the dataset and display its information. Preprocess the data (drop null values etc.) Convert the categorical values into numeric format. Apply the apriori algorithm on the above dataset to generate the frequent itemsets and association rules. (Use Market basket dataset)

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules

df = pd.read_csv('Market_Basket_Optimisation.csv', header=None)

print("Dataset Info:")

print(df.info())

print("\nFirst 5 rows:")

print(df.head())

transactions = []

for i in range(len(df)):

    transaction = [str(df.iloc[i,j]) for j in range(df.shape[1]) if
pd.notna(df.iloc[i,j])]

    transactions.append(transaction)

te = TransactionEncoder()

te_array = te.fit(transactions).transform(transactions)

df_encoded = pd.DataFrame(te_array, columns=te.columns_)

freq_items = apriori(df_encoded, min_support=0.05, use_colnames=True)

freq_items['length'] = freq_items['itemsets'].apply(lambda x: len(x))

print("\nFrequent Itemsets:")

print(freq_items)

rules = association_rules(freq_items, metric='support', min_threshold=0.05)

rules = rules.sort_values(['support','confidence'], ascending=[False,False])
```

```
print("\nAssociation Rules:")  
print(rules)
```

3. Write a python program to read the dataset and display its information. Preprocess the data (drop null values etc.) Convert the categorical values into numeric format. Apply the apriori algorithm on the above dataset to generate the frequent itemsets and association rules. (Use groceries dataset)

```
import pandas as pd  
from mlxtend.preprocessing import TransactionEncoder  
from mlxtend.frequent_patterns import apriori, association_rules  
df = pd.read_csv('Groceries_dataset.csv')  
print("Dataset Info:")  
print(df.info())  
print("\nFirst 5 rows:")  
print(df.head())  
df.drop('Date', axis=1, inplace=True)  
df = df.dropna()  
df_grouped=df.groupby('Member_number')['itemDescription'].apply(list).reset_  
index(name='Basket')  
transactions = df_grouped['Basket'].tolist()  
te = TransactionEncoder()  
te_array = te.fit(transactions).transform(transactions)  
df_encoded = pd.DataFrame(te_array, columns=te.columns_)  
freq_items = apriori(df_encoded, min_support=0.05, use_colnames=True)  
freq_items['length'] = freq_items['itemsets'].apply(lambda x: len(x))  
print("\nFrequent Itemsets:")  
print(freq_items)
```

```

rules = association_rules(freq_items, metric='support', min_threshold=0.05)
rules = rules.sort_values(['support','confidence'], ascending=[False,False])
print("\nAssociation Rules:")
print(rules)

```

5. Create the dataset in python & Convert the categorical values into numeric format. Apply the apriori algorithm on the dataset to generate the frequent itemsets and association rules. Repeat the process with different min_sup values.

TID	Items
1	butter, bread, milk
2	butter, flour, milk, sugar
3	butter, eggs, milk, salt
4	eggs
5	butter, flour, milk, salt

```

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
# Step 1: Define transactions based on your table
transactions = [
    ['butter', 'bread', 'milk'],
    ['butter', 'flour', 'milk', 'sugar'],
    ['butter', 'eggs', 'milk', 'salt'],
    ['eggs'],
    ['butter', 'flour', 'milk', 'salt']
]
# Step 2: Encode transactions into 0/1 format
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)

```

```

df = pd.DataFrame(te_array, columns=te.columns_)
print("Encoded Transactions:")
print(df)
# Step 3: Apply Apriori algorithm
freq_items = apriori(df, min_support=0.6, use_colnames=True) # Example
min_support=0.6
freq_items['length'] = freq_items['itemsets'].apply(lambda x: len(x))
print("\nFrequent Itemsets:")
print(freq_items)
# Step 4: Generate association rules
rules = association_rules(freq_items, metric='support', min_threshold=0.05)
rules = rules.sort_values(['support','confidence'], ascending=[False, False])
print("\nAssociation Rules:")
print(rules)

```

6. Create the dataset in python & Convert the categorical values into numeric format. Apply the apriori algorithm on the dataset to generate the frequent itemsets and association rules. Repeat the process with different min_sup values.

TID	Items
1	{Apple , Mango , Banana}
2	{Mango , Banana , Cabbage , Carrots}
3	{Mango , Banana ,Carrots}
4	{Mango , Carrots}

```

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
# Step 1: Define transactions from the table

```

```

transactions = [
    ['Apple', 'Mango', 'Banana'],
    ['Mango', 'Banana', 'Cabbage', 'Carrots'],
    ['Mango', 'Banana', 'Carrots'],
    ['Mango', 'Carrots']
]

# Step 2: Encode transactions into 0/1 numeric format
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)
print("Encoded Transactions:")
print(df)
print("\nColumns:", te.columns_)

# Step 3: Apply Apriori algorithm
freq_items = apriori(df, min_support=0.5, use_colnames=True)
freq_items['length'] = freq_items['itemsets'].apply(lambda x: len(x))
print("\nFrequent Itemsets:")
print(freq_items)

# Step 4: Generate association rules
rules = association_rules(freq_items, metric='support', min_threshold=0.05)
rules = rules.sort_values(['support', 'confidence'], ascending=[False, False])
print("\nAssociation Rules:")
print(rules)

```

7. Create the dataset in python & Convert the categorical values into numeric format. Apply the apriori algorithm on the dataset to generate the frequent itemsets and association rules. Repeat the process with different min_sup values.

	company	model	year
0	Tata	Nexon	2017
1	MG	Astor	2021
2	KIA	Seltos	2019
3	Hyundai	Creta	2015

```

import pandas as pd

from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder

# Step 1: Define transactions from the table in the image
transactions = [

    ['Tata', 'Nexon', '2017'],
    ['MG', 'Astor', '2021'],
    ['KIA', 'Seltos', '2019'],
    ['Hyundai', 'Creta', '2015']

]

# Step 2: Encode transactions into 0/1 numeric format
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)
print("Encoded Transactions:")
print(df)
print("\nColumns:", te.columns_)

# Step 3: Apply Apriori algorithm with min_support=0.5
freq_items = apriori(df, min_support=0.25, use_colnames=True)
freq_items['length'] = freq_items['itemsets'].apply(lambda x: len(x))
print("\nFrequent Itemsets (min_support=0.5):")
print(freq_items)

# Step 4: Generate association rules

```

```

rules = association_rules(freq_items, metric='support', min_threshold=0.05)
rules = rules.sort_values(['support','confidence'], ascending=[False, False])
print("\nAssociation Rules (min_support=0.5):")
print(rules)

# Step 5: Repeat with a different min_support (e.g., 0.25)
freq_items_low = apriori(df, min_support=0.25, use_colnames=True)
freq_items_low['length'] = freq_items_low['itemsets'].apply(lambda x: len(x))
print("\nFrequent Itemsets (min_support=0.25):")
print(freq_items_low)

rules_low = association_rules(freq_items_low, metric='support',
min_threshold=0.05)

rules_low = rules_low.sort_values(['support','confidence'], ascending=[False,
False])

print("\nAssociation Rules (min_support=0.25):")
print(rules_low)

```

8. Create the following dataset in python & Convert the categorical values into numeric format. Apply the apriori algorithm on the above dataset to generate the frequent itemsets and association rules. Repeat the process with different min_sup values.

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

```

import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules

```

```

dataset = [
    ['Bread', 'Milk'],
    ['Bread', 'Diaper', 'Beer', 'Eggs'],
    ['Milk', 'Diaper', 'Beer', 'Coke'],
    ['Bread', 'Milk', 'Diaper', 'Beer'],
    ['Bread', 'Milk', 'Diaper', 'Coke']
]

# Transaction Encoding
te = TransactionEncoder()
te_array = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_array, columns=te.columns_)

# Generate Frequent Itemsets
freq_items = apriori(df, min_support=0.5, use_colnames=True)
print(freq_items)

# Generate Association Rules
rules = association_rules(freq_items, metric='support', min_threshold=0.05)
rules = rules.sort_values(['support', 'confidence'], ascending=[False, False])
print(rules)

```

Text Analysis

1. Consider any text paragraph. Preprocess the text to remove any special characters and digits. Generate the summary using extractive summarization process.

```

import nltk
import re
import heapq
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize

```

text = " Text Summarization is an important application of Natural Language Processing. It is used to reduce large amounts of text into shorter versions while preserving the main meaning of the content. There are two main types of summarizations: extractive and abstractive summarization."

```
clean_text = re.sub(r'^a-zA-Z\s', "", text)
```

```
print("Cleaned Text:\n", clean_text)
```

```
# Tokenize into sentences
```

```
sentences = sent_tokenize(text)
```

```
print("\nSentences:\n", sentences)
```

```
# Word tokenization and stopwords removal
```

```
stop_words = set(stopwords.words('english'))
```

```
word_frequencies = {}
```

```
for word in word_tokenize(clean_text.lower()):
```

```
    if word not in stop_words:
```

```
        if word not in word_frequencies:
```

```
            word_frequencies[word] = 1
```

```
        else:
```

```
            word_frequencies[word] += 1
```

```
# Normalize word frequencies
```

```
max_frequency = max(word_frequencies.values())
```

```
for word in word_frequencies:
```

```
    word_frequencies[word] /= max_frequency
```

```
# Score sentences
```

```
sentence_scores = {}
```

```
for sentence in sentences:
```

```
    for word in word_tokenize(sentence.lower()):
```

```
        if word in word_frequencies:
```

```
            if sentence not in sentence_scores:
```

```

        sentence_scores[sentence] = word_frequencies[word]
    else:
        sentence_scores[sentence] += word_frequencies[word]

# Select top 2 sentences as summary
summary_sentences = heapq.nlargest(2, sentence_scores,
key=sentence_scores.get)
summary = ''.join(summary_sentences)
print("\nOriginal Text:\n", text)
print("\nSummary:\n", summary)

```

2./4. Consider any text paragraph. Remove the stopwords. Tokenize the paragraph to extract words and sentences. Calculate the word frequency distribution and plot the frequencies.

```

import nltk
import string
import matplotlib.pyplot as plt
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.probability import FreqDist

# Sample paragraph
text = """
Natural Language Processing is a fascinating field of artificial intelligence.
It helps computers understand, interpret, and generate human language. NLP is
widely used in chatbots, machine translation, sentiment analysis, and many
other applications.
"""

# Sentence Tokenization
sentences = sent_tokenize(text)
print("Sentences:")

```

```

print(sentences)

# Word Tokenization
words = word_tokenize(text)
print("\nWords:")
print(words)

# Remove Stopwords & Punctuation
stop_words = set(stopwords.words('english'))
filtered_words = [
    word.lower() for word in words
    if word.lower() not in stop_words and word not in string.punctuation ]
print("\nFiltered Words (No Stopwords):")
print(filtered_words)

# Frequency Distribution
freq_dist = FreqDist(filtered_words)
print("\nWord Frequency Distribution:")
print(freq_dist)

# Plot Frequency Distribution
plt.figure()
freq_dist.plot(10)
plt.title("Top 10 Word Frequencies")
plt.show()

```

3. Consider text paragraph. “*So, keep working. Keep striving. Never give up. Fall down seven times, get up eight. Ease is a greater threat to progress than hardship. Ease is a greater threat to progress than hardship. So, keep moving, keep growing, keep learning. See you at work.*” Preprocess the text to remove any special characters and digits. Generate the summary using extractive summarization process.

```
import nltk
```

```

import re
import heapq
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize

text = """So, keep working. Keep striving. Never give up. Fall down seven
times, get up eight.

Ease is a greater threat to progress than hardship. Ease is a greater threat to
progress than hardship.

So, keep moving, keep growing, keep learning. See you at work."""

# 1. Preprocess the text: Remove special characters and digits
clean_text = re.sub(r'^a-zA-Z\s', "", text)
print("Cleaned Text:\n", clean_text)

# 2. Sentence tokenization
sentences = sent_tokenize(text)
print("\nSentences:\n", sentences)

# 3. Word tokenization and stopwords removal
stop_words = set(stopwords.words('english'))
word_frequencies = {}

for word in word_tokenize(clean_text.lower()):
    if word not in stop_words:
        if word not in word_frequencies:
            word_frequencies[word] = 1
        else:
            word_frequencies[word] += 1

# 4. Normalize word frequencies
max_frequency = max(word_frequencies.values())

for word in word_frequencies:
    word_frequencies[word] /= max_frequency

```

```

# 5. Score sentences based on word frequencies
sentence_scores = {}
for sentence in sentences:
    for word in word_tokenize(sentence.lower()):
        if word in word_frequencies:
            if sentence not in sentence_scores:
                sentence_scores[sentence] = word_frequencies[word]
            else:
                sentence_scores[sentence] += word_frequencies[word]

# 6. Select top 2 sentences as summary
summary_sentences = heapq.nlargest(2, sentence_scores,
key=sentence_scores.get)
summary = ''.join(summary_sentences)

# 7. Display results
print("\nOriginal Text:\n", text)
print("\nSummary:\n", summary)

```

5. Consider text paragraph. “*Hello all, Welcome to Python Programming Academy. Python Programming Academy is a nice platform to learn new programming skills. It is difficult to get enrolled in this Academy.*” Remove the stopwords. Tokenize the paragraph to extract words and sentences. Calculate the word frequency distribution.

```

import nltk
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.probability import FreqDist
# Sample text
text = """
Hello all, Welcome to Python Programming Academy.
Python Programming Academy is a nice platform to learn new programming
skills.

```

It is difficult to get enrolled in this Academy.

"""

1. Sentence tokenization

sentences = sent_tokenize(text)

print("Sentences:")

print(sentences)

2. Word tokenization

words = word_tokenize(text)

print("\nWords:")

print(words)

3. Remove stopwords and punctuation

stop_words = set(stopwords.words('english'))

filtered_words = [

word.lower() for word in words

if word.lower() not in stop_words and word not in string.punctuation

]

print("\nFiltered Words (No Stopwords or Punctuation):")

print(filtered_words)

4. Word Frequency Distribution

freq_dist = FreqDist(filtered_words)

print("\nWord Frequency Distribution:")

for word, freq in freq_dist.items():

print(f"{word}: {freq}")

6. Consider the following review messages. Perform sentiment analysis on the messages. i. I purchased headphones online. I am very happy with the product. ii. I saw the movie yesterday. The animation was really good but the script was ok.

import nltk

from nltk.sentiment.vader import SentimentIntensityAnalyzer

Download VADER lexicon

nltk.download('vader_lexicon')

Initialize VADER Sentiment Analyzer

vader_analyzer = SentimentIntensityAnalyzer()

List of review messages

```

reviews = [
    "I purchased headphones online. I am very happy with the product.",
    "I saw the movie yesterday. The animation was really good but the script was ok."
]

# Analyze sentiment for each review
for i, text in enumerate(reviews, start=1):
    print(f"\nReview {i}: {text}")
    result = vader_analyzer.polarity_scores(text)
    # Print Positive, Negative, Neutral percentages
    print("Positive: {:.2f}%".format(result['pos']*100))
    print("Negative: {:.2f}%".format(result['neg']*100))
    print("Neutral: {:.2f}%".format(result['neu']*100))
    # Determine overall sentiment
    if result['compound'] >= 0.05:
        print("Overall Sentiment: Positive")
    elif result['compound'] <= -0.05:
        print("Overall Sentiment: Negative")
    else:
        print("Overall Sentiment: Neutral")

```

7. Consider the following review messages. Perform sentiment analysis on the messages. i. I enjoy listening to music ii. I take a walk in the park everyday

```

import nltk

from nltk.sentiment.vader import SentimentIntensityAnalyzer

# Download VADER lexicon
nltk.download('vader_lexicon')

```

```

# Initialize Sentiment Analyzer
vader_analyzer = SentimentIntensityAnalyzer()

# List of review messages
reviews = [
    "I enjoy listening to music",
    "I take a walk in the park everyday"
]

# Analyze sentiment for each review
for i, text in enumerate(reviews, start=1):
    print(f"\nReview {i}: {text}")
    result = vader_analyzer.polarity_scores(text)
    # Print Positive, Negative, Neutral percentages
    print("Positive: {:.2f}%".format(result['pos']*100))
    print("Negative: {:.2f}%".format(result['neg']*100))
    print("Neutral: {:.2f}%".format(result['neu']*100))
    # Determine overall sentiment
    if result['compound'] >= 0.05:
        print("Overall Sentiment: Positive")
    elif result['compound'] <= -0.05:
        print("Overall Sentiment: Negative")
    else:
        print("Overall Sentiment: Neutral")

```

8. Consider the following review messages. Perform sentiment analysis on the messages. i. "I absolutely love this product. It works perfectly and makes my life easier!" ii. "I am very disappointed with this service. It was slow and not helpful at all."

```
import nltk
```

```
from nltk.sentiment.vader import SentimentIntensityAnalyzer

# Download VADER lexicon
nltk.download('vader_lexicon')

# Initialize VADER Sentiment Analyzer
vader_analyzer = SentimentIntensityAnalyzer()

# List of review messages
reviews = [
    "I absolutely love this product. It works perfectly and makes my life easier!",
    "I am very disappointed with this service. It was slow and not helpful at all."
]

# Analyze sentiment for each review
for i, text in enumerate(reviews, start=1):
    print(f"\nReview {i}: {text}")
    result = vader_analyzer.polarity_scores(text)
    # Print Positive, Negative, Neutral percentages
    print("Positive: {:.2f}%".format(result['pos']*100))
    print("Negative: {:.2f}%".format(result['neg']*100))
    print("Neutral: {:.2f}%".format(result['neu']*100))
    # Determine overall sentiment
    if result['compound'] >= 0.05:
        print("Overall Sentiment: Positive")
    elif result['compound'] <= -0.05:
        print("Overall Sentiment: Negative")
    else:
        print("Overall Sentiment: Neutral")
```

